Vidyavardhini's College of Engineering & Technology
Department of Computer Science Engineering (Data Science)

Name:	**Jay Chandrasen Talekar**
Roll No:	**50**
Class/Sem:	TE/VI
Experiment No.:	7
Title:	To study and implement designing, scheduling, and monitoring workflows and data pipelines using Apache Airflow.
Date of Performance:	
Date of Submission:	
Marks:	
Sign of Faculty:	

Aim:

To study and implement designing, scheduling, and monitoring workflows and data pipelines using Apache Airflow.

Theory:

What is Apache Airflow?

Apache Airflow is an open-source platform used to develop, schedule, and monitor workflows and data pipelines.

What is a DAG?

A DAG (Directed Acyclic Graph) is a collection of tasks arranged in a specific order. It defines how tasks in a workflow depend on each other.

What is a Task?

A task is an individual unit of work in an Airflow workflow. Tasks can perform operations such as running Python code or shell commands.

What is Scheduling?

Scheduling allows Airflow to automatically execute a workflow at a specified time or interval.

What is Monitoring?

Airflow provides a web interface to monitor workflows, task status, execution history, and logs.

Features of Apache Airflow:

- It is an open-source workflow management platform.
- It allows workflows to be scheduled automatically.
- It provides a web interface for monitoring workflows.
- It supports task dependencies.
- It provides logs for troubleshooting tasks.

Steps to Design, Schedule and Monitor a Workflow Using Airflow:

Step 1: Install Apache Airflow

Install Airflow using pip:

```
pip install apache-airflow
```

Step 2: Initialize Airflow

Initialize the Airflow database:

```
airflow db migrate
```

Step 3: Create a DAG File

Create a file named:

```
simple_dag.py
```

Add the following simple workflow:

```
from datetime import datetime
from airflow import DAG
from airflow.operators.python import PythonOperator

def task1():
    print("Task 1 executed")

def task2():
    print("Task 2 executed")

with DAG(
    dag_id="simple_dag",
    start_date=datetime(2024, 1, 1),
    schedule="@daily",
    catchup=False
) as dag:

    t1 = PythonOperator(
        task_id="task_1",
        python_callable=task1
    )

    t2 = PythonOperator(
        task_id="task_2",
        python_callable=task2
    )

    t1 >> t2
```

Step 4: Start Airflow

Start the Airflow web server:

```
airflow standalone
```

Airflow will start its services and provide a web interface. Open the displayed Airflow URL in a browser.

Step 5: View the DAG

Open the Airflow web interface and find the DAG named simple_dag. Enable the DAG to allow it to run.

Step 6: Run the DAG

Trigger the DAG manually from the Airflow web interface.

The tasks will execute according to their dependency:

```
Task 1  →  Task 2
```

Step 7: Monitor the Workflow

Open the DAG in the Airflow web interface. The status of each task can be viewed. Click on a task to view its logs and execution details.

Step 8: Check Task Logs

Select a task and open its logs. The output generated by the Python function will be displayed.

Step 9: Schedule the Workflow

The DAG is configured with a daily schedule using:

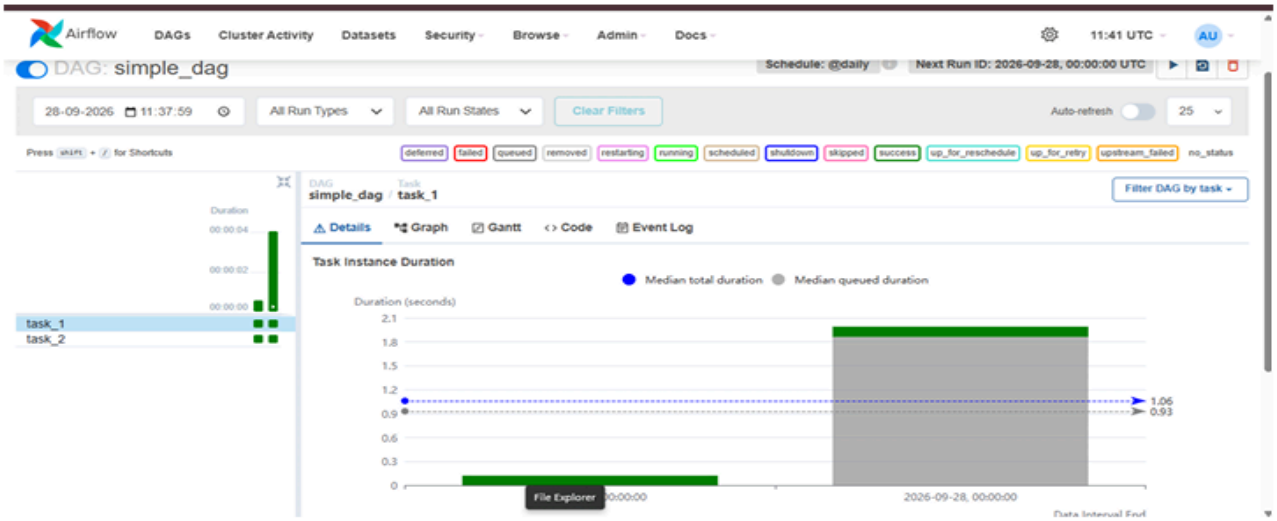
```
schedule="@daily"
```

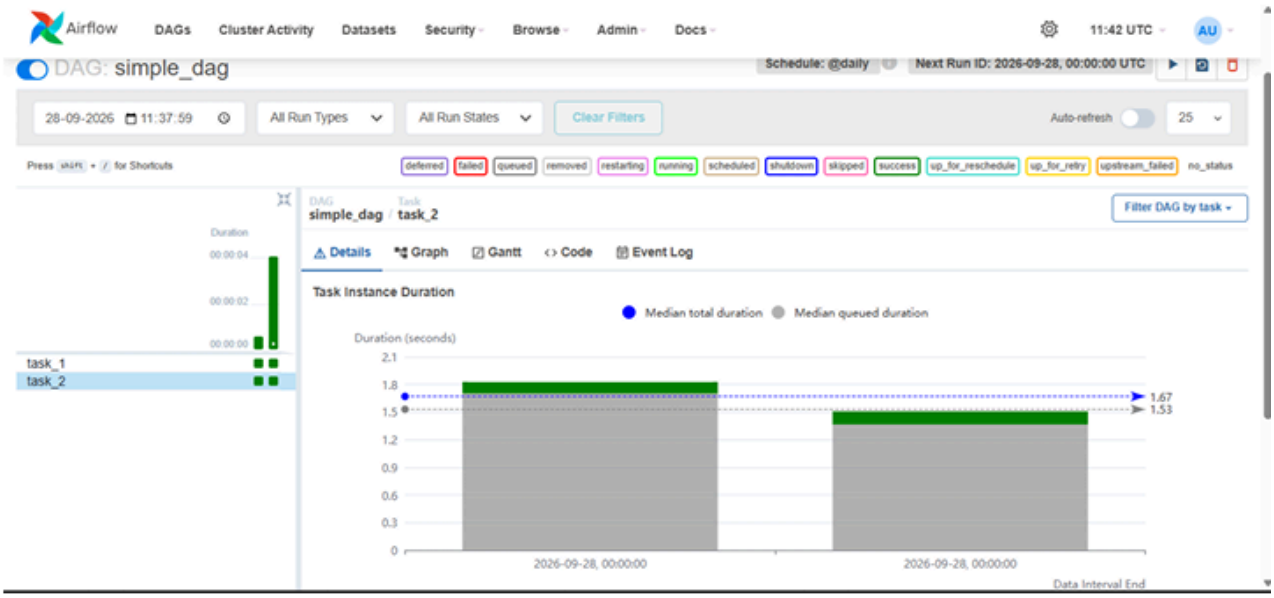
Airflow will schedule the workflow according to the defined schedule.

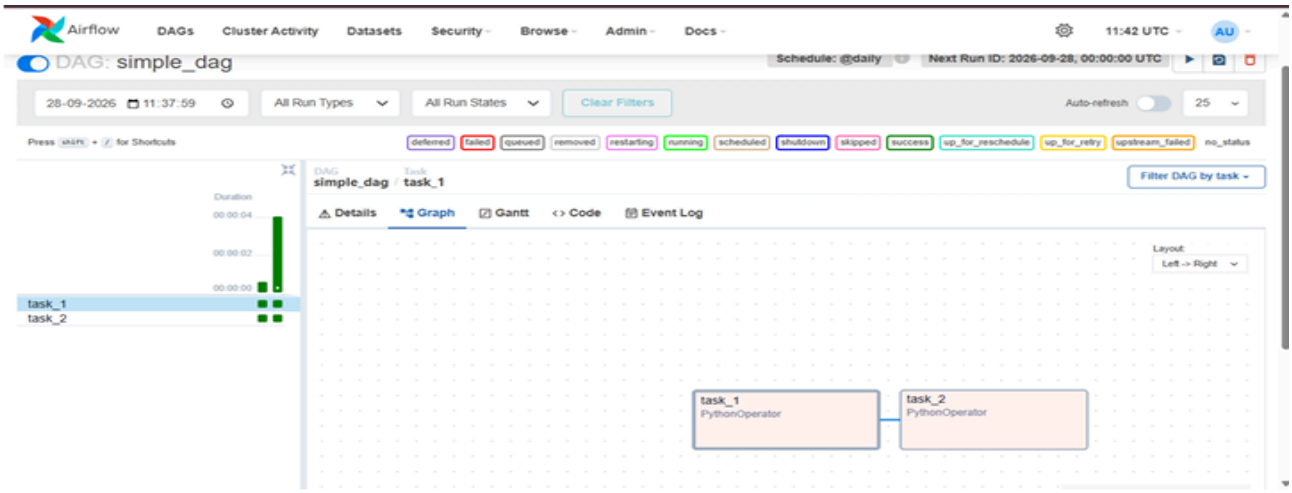
Step 10: Stop Airflow

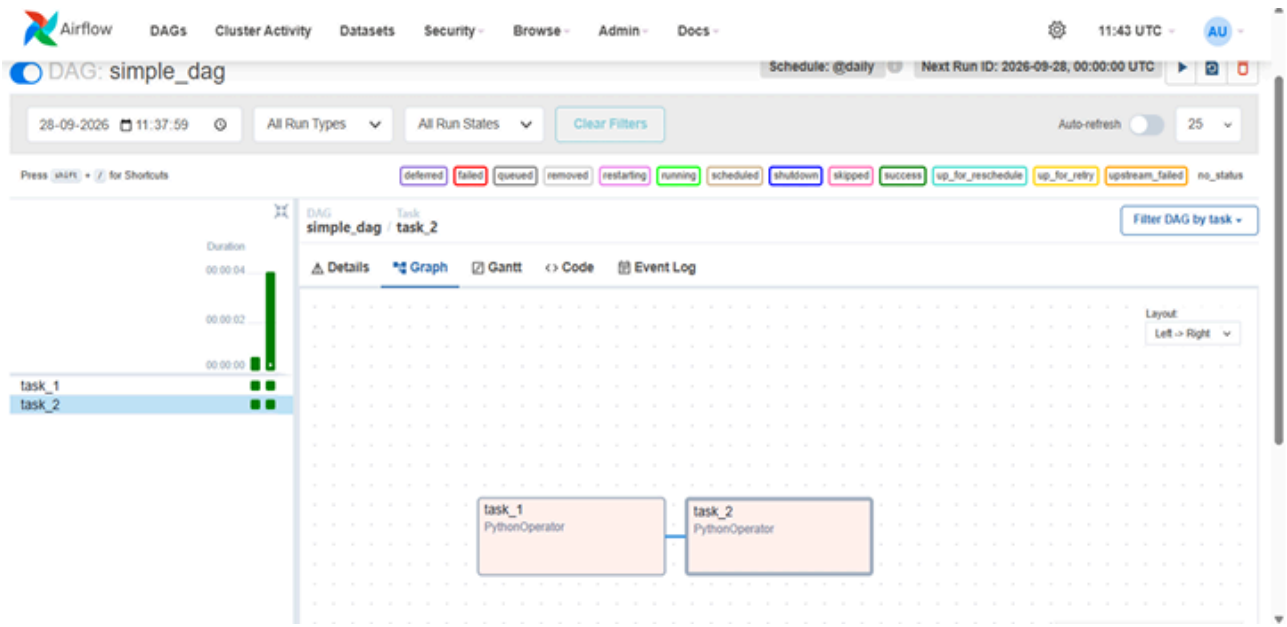
After completing the experiment, stop the Airflow services using Ctrl + C in the terminal where they are running.

Output -




Airflow
DAGs
Cluster Activity
Datasets
Security
Browse
Admin
Docs
11:42 UTC
AU
DAG: simple_dag
Schedule: @daily
Next Run ID: 2026-09-28, 00:00:00 UTC
28-09-2026 11:37:59
All Run Types
All Run States
Clear Filters
Auto-refresh
25
Press shift + / for Shortcuts
deferred
failed
queued
removed
restarting
running
scheduled
shutdown
skipped
success
up_for_reschedule
up_for_retry
upstream_failed
no_status
Duration
00:00:04
00:00:02
00:00:00
task_1
task_2
DAG
simple_dag
Task
task_2
Filter DAG by task
Details
Graph
Gantt
Code
Event Log
Task Instance Duration
Median total duration
Median queued duration
Duration (seconds)
2.1
1.8
1.5
1.2
0.9
0.6
0.3
0
1.67
1.53
2026-09-28, 00:00:00
2026-09-28, 00:00:00
Data Interval End


Airflow
DAGs
Cluster Activity
Datasets
Security
Browse
Admin
Docs
11:42 UTC
AU
DAG: simple_dag
Schedule: @daily
Next Run ID: 2026-09-28, 00:00:00 UTC
28-09-2026 11:37:59
All Run Types
All Run States
Clear Filters
Auto-refresh
25
Press shift + / for Shortcuts
deferred
failed
queued
removed
restarting
running
scheduled
shutdown
skipped
success
up_for_reschedule
up_for_retry
upstream_failed
no_status
Duration
00:00:04
00:00:02
00:00:00
task_1
task_2
DAG
simple_dag
Task
task_1
Filter DAG by task
Details
Graph
Gantt
Code
Event Log
Layout:
Left -> Right
task_1
PythonOperator
task_2
PythonOperator


Airflow
DAGs
Cluster Activity
Datasets
Security
Browse
Admin
Docs
11:43 UTC
AU
DAG: simple_dag
Schedule: @daily
Next Run ID: 2026-09-28, 00:00:00 UTC
28-09-2026 11:37:59
All Run Types
All Run States
Clear Filters
Auto-refresh
25
Press shift + / for Shortcuts
deferred
failed
queued
removed
restarting
running
scheduled
shutdown
skipped
success
up_for_reschedule
up_for_retry
upstream_failed
no_status
Duration
00:00:04
00:00:02
00:00:00
task_1
task_2
DAG
simple_dag
Task
task_2
Filter DAG by task
Details
Graph
Gantt
Code
Event Log
Layout:
Left -> Right
task_1
PythonOperator
task_2
PythonOperator

Conclusion:

1. **What is Apache Airflow?**
   Apache Airflow is an open-source platform used to create, schedule, and monitor workflows.

2. **What is a DAG in Airflow?**
   A DAG (Directed Acyclic Graph) is a collection of tasks arranged in a specific order with defined dependencies. It represents a workflow.

3. **What is a task?**
   A task is a single unit of work in an Airflow workflow. In our experiment, `task_1` and `task_2` are examples of tasks.

4. **What is the purpose of scheduling in Airflow?**
   Scheduling is used to define when a workflow should run automatically. In our experiment, `@daily` is used to schedule the DAG daily.

5. **How can a workflow be monitored in Airflow?**
   A workflow can be monitored through the Airflow Web UI, where we can view the DAG status, task status, and execution history.

6. **What is the use of task dependencies?**
   Task dependencies define the order in which tasks should execute. In our experiment, Task 1 must execute before Task 2.

7. **Where can the logs of a task be viewed?**
   The logs of a task can be viewed in the Airflow Web UI by opening the task and selecting the **Logs** option.